3. Jednosmerné asociácie (a kolekcie)
3.1 Kolekcia objektov jednoduchého dátového typu
3.2 Jednosmerná N:N asociácia
3.3 Jednosmerná N:1 asociácia
3.4 Jednosmerná 1:N asociácia
Cvičenia

3. Jednosmerné asociácie (a kolekcie)

Prednášky ilustrujú na jednoduchých projektoch, akým spôsobom mapuje ORM jednotlivé typy asociácií na relačnú databázu.
Pri výklade a implementácii postupujeme nasledovne:

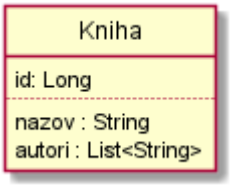
- Implementujeme entitné triedy na základe OO-modelu (UML class diagramu).
- Implementujeme aplikáciu s funkciou na vkladanie údajov do databázy a príp. ďalšími funkciami na dotazovanie.
- Databázu vygenerujeme z entitných tried pomocou JPA
  (nastavíme schema-generation `create` a spustíme testovaciu aplikáciu).
- Pozrieme si štruktúru a dátový obsah vytvorených tabuliek.
- Príp. na verziách aplikácie ukážeme ďalšie vlastnosti a fukcionalitu.

3.1 Kolekcia objektov jednoduchého dátového typu

Pripomeňme, že prvá normálna forma pre relačné databázy vyžaduje, aby stĺpce obsahovali len atomické hodnoty. Teda hodnoty nesmú byť štruktúrované alebo viacnásobné. Na rozdiel od toho, v objektovo orientovaných jazykoch dátové členy tried nemusia obsahovať len hodnoty jednoduchých dátových typov, ale aj štruktúry alebo kolekcie.

Nasledujúci príklad ilustruje, akým spôsobom JPA mapuje viacnásobný dátový člen jednoduchého dátového typu. Pozri projekt p31kolekcia

Príklad: Kniha so zoznamom mien autorov.



Entitná trieda - verzia 1.

```
@Entity
public class Kniha implements Serializable {
  @Id
  private Long id;
  private String nazov;
  private List<String> autori;
```

Aplikácia

Tabuľky vytvoríme spustením aplikácie, ktorá vytvorí inštanciu knihy s viacerými autormi a vloží ju do databázy (stratégia na generovanie schémy: `create`).

Všimnite si, že vo vygenerovanej tabuľke `KNIHA` je stĺpec `AUTORI` typu `BLOB`, t.j. obsahuje binárne dáta - serializovaný zoznam autorov.
Jednotlivé položky zoznamu však nebudú prístupné pre štandardného DB-klienta.
Poznámka: Problém ilustruje Cvičenie 1.

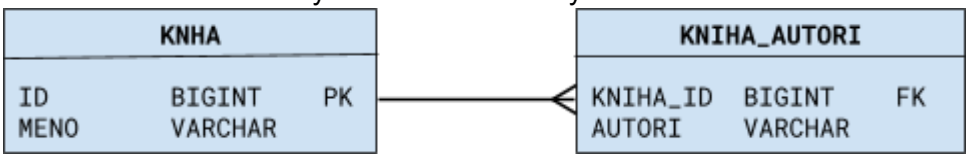
Entitná trieda - verzia 2.

Zoznamu autorov dáme anotáciu `ElementCollection`

```
  @ElementCollection
  private List<String> autori;
```

Databáza

Tentokrát budú vytvorené dve tabuľky:

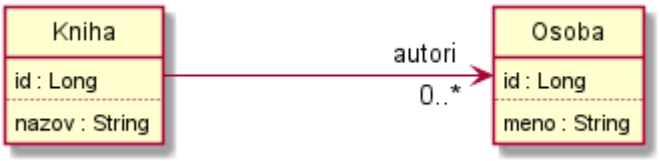


3.2 Jednosmerná N:N asociácia

N:N asociáciu ilustrujeme na modifikácii predchádzajúceho príkladu, kde budú autori inštancie entitnej triedy `Osoba`. Pozri projekt p32uniNN.

Príklad: Kniha so zoznamom (referencií na) autorov.

Kniha môže mať viac autorov a jedna osoba môže byť autorom viacerých kníh. Kniha má zoznam svojich autorov, ale autori nemajú zoznam svojich kníh.



Entitné triedy

```
@Entity
public class Kniha implements Serializable {
  @Id
  private Long id;
  private String nazov;
  private List<Osoba> autori;
```

```
@Entity
public class Osoba implements Serializable {
  @Id
  private Long id;
  private String meno;
```

Poznámka: skutočnosť, že asociácia je N:N môžeme vyjadriť anotáciou `@ManyToMany`. V tomto prípade ju však netreba uviesť, pretože je to default pre kolekcie.

```
  @ManyToMany
  private List<Osoba> autori;
```

Aplikácia

Vytvoríme inštanciu knihy s viacerými autormi a vložíme ju do databázy.

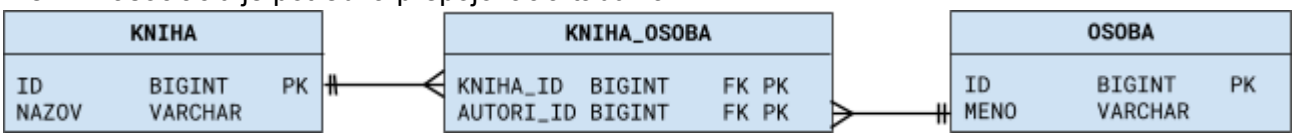
Dôležité:Metódu `persist` treba zavolať aj pre všetky novovytvorené asociované objekty a urobiť `commit` v tej istej transakcii.

```
      kniha.getAutori().add(autor1);
      kniha.getAutori().add(autor2);

      em.getTransaction().begin();
      em.persist(kniha);
      em.persist(autor1);
      em.persist(autor2);
      em.getTransaction().commit();
```

Relačný diagram

Pre N:N asociáciu je potrebná prepojovacia tabuľka.

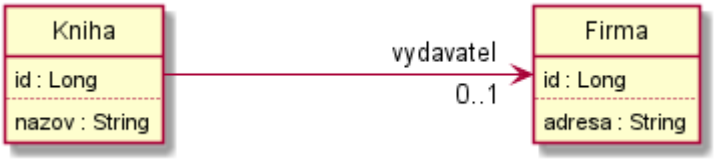


Všimnite si, ako je vytvorený názov prepojovacej tabuľky a jej stĺpcov. Anotáciou `@JoinTable` môžeme tieto názvy definovať sami, pozri cvičenie 3.2.

3.3 Jednosmerná N:1 asociácia

Príkladom jednosmernej asociácie N:1 je vzťah medzi knihou a jej vydavateľom. Trieda `Kniha` máreferenciu na firmu, ktorá ju vydáva. Trieda `Firma` však nemá referencie na svoje knihy. Pozri projekt p33uniN1

Príklad: Kniha s referenciou na jej vydavateľa.



Poznámka: Kardinalita 0..1 na strane vydavateľa musí byť vyznačená, pretože inak by nebolo z diagramu jasné, že jedna kniha môže mať len jedného vydavateľa.

Entitné triedy

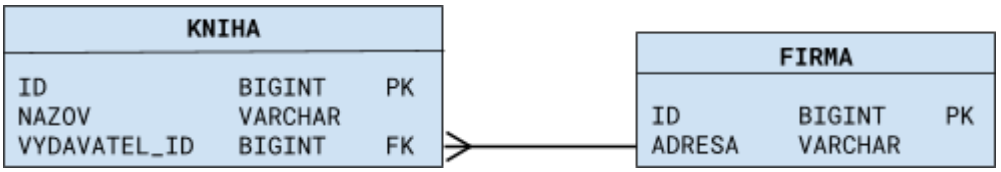
```
@Entity
public class Kniha implements Serializable {
  @Id
  private Long id;
  private String nazov;
  private Firma vydavatel;
```

```
@Entity
public class Firma implements Serializable {
  @Id
  private Long id;
  private String adresa;
```

Poznámka: skutočnosť, že asociácia je N:1 môžeme vyjadriť anotáciou `@ManyToOne`. (v tomto prípade však v tomto prípade nutná, pretože je to default pre jednoduché referencie)

```
  @ManyToOne
  private Firma vydavatel;
```

Relačný diagram vygenerovaných tabuliek



3.4 Jednosmerná 1:N asociácia

Otočením orientácie asociácie v predchádzajúcom príklade dostaneme jednosmernú 1:N asociáciu. Tentoraz má teda firma zoznam publikácií, ktoré vydáva, ale kniha nepozná svojho vydavateľa. Pozri projekt p34uni1N.

Príklad: Vydavateľská firma so zoznamom publikácií, ktoré vydáva.

Poznámky k diagramu tried:

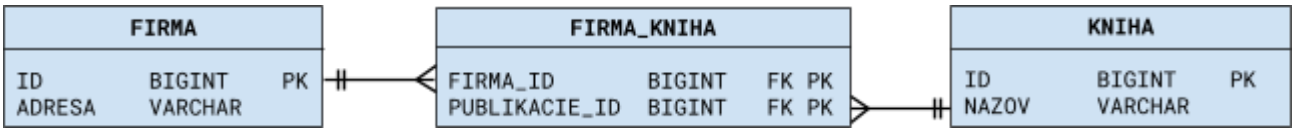
- Hoci trieda Kniha nemá referenciu na Firmu, kardinalita 0..1 na strane firmy je potrebná, aby bolo jasné, že ide o 1:N asociáciu (nie N:N), teda Kniha nemôže mať viac vydavateľov.
- Na strane knihy nie je uvedená kardinalita, to znamená, že je ľubovoľná teda je 0..*.

Entitné triedy - verzia 1.

```
@Entity
public class Kniha implements Serializable {
  @Id
  private Long id;
  private String nazov;
```

```
@Entity
public class Firma implements Serializable {
  @Id
  private Long id;
  private String adresa;
  private List<Kniha> publikacie;
```

Relačný diagram vygenerovaných tabuliek



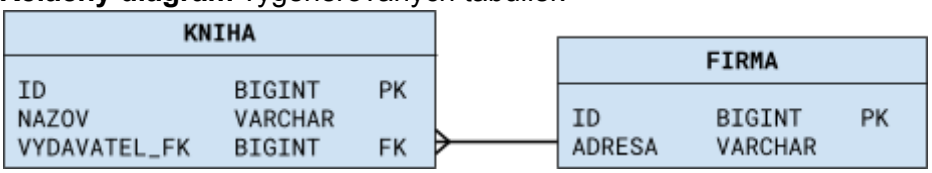
Poznámka: Bola opäť vygenerovaná prepojovacia tabuľka, pretože len na základe entitných tried nie je možné určiť, či asociácia mala byť 1:N alebo N:N (t.j. či jedna kniha nemôže mať viac vydavateľov)

Entitné triedy - verzia 2.

```
@Entity
public class Firma implements Serializable {
  @Id
  private Long id;
  private String adresa;
  @OneToMany
  @JoinColumn(name = "VYDAVATEL_FK")
  private List<Kniha> publikacie;
```

Ak nechceme vytvoriť prepojovaciu tabuľku, keďže ide o 1:N asociáciu, použijeme anotáciou `@OneToMany`. Okrem toho však treba **definovať názov prepojovacieho stĺpca**, ktorý bude v tabuľke asociovanej triedy. To určíme **anotáciou `@JoinColumn`.**

Relačný diagram vygenerovaných tabuliek



Poznámky: Všimnite si, že teraz má databáza rovnakú štruktúru ako v prípade N:1 asociácie (predchádzajúci projekt). Pri 1:N aj N:1 asociáciách je cudzí kľúč v databáze vždy v tabuľke na viacnásobnej strane a referencuje entitu na jednoduchej strane asociácie.

Jediný rozdiel je tu v názve FK stĺpca, ktorý sme nazvali `VYDAVATEL_FK`, avšak v prípade N:1 bol vygenerovaný názov `VYDAVATEL_ID`.

Cvičenia

V nasledujúcich úlohách vytvárame rôzne modifikácie projektov z prednášky.

Cvičenie 1. Doplňte projekt p31kolekcia o dve funkcie:

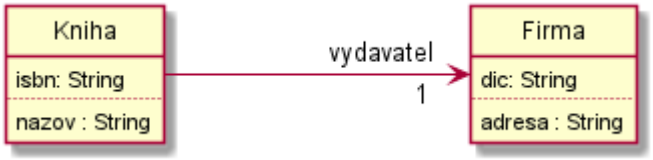
1. vyhľadá a načíta knihu podľa `id` (pomocou `find`) a vypíše jej autorov.
2. pomocou JPQL dotazu vyhľadá knihy podľa autora.
   Návod: skúste dotaz: `"select k from Kniha k where k.autori='Ritchey'"`

Implementujte aplikáciu na testovanie funkcií. Overte, ako pracujú pre obe verzie entitnej triedy, s anotáciou `@ElementCollection` a aj bez nej.

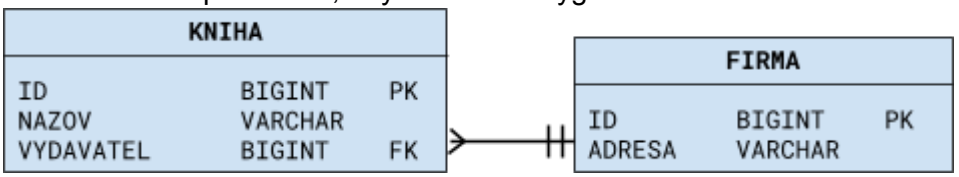
Cvičenie 2. V projekte p32uniNN upravte entitnú triedu pomocou anotácie `@JoinTable` (pozri https://www.baeldung.com/jpa-many-to-many) tak, aby vygenerovaná prepojovacia tabuľka mala štruktúru:

Poznámka: Názvy tried a dátových členov pri tom meniť netreba.

Cvičenie 3. Modifikujte projekt p33uniN1 podľa upraveného UML diagramu:



Anotácie tried upravte tak, aby z nich boli vygenerované databázové tabuľky podľa diagramu:



a) Vydavateľ bol povinný atribút, teda vo vygenerovanej tabuľke by mal byť FK stĺpec notNull (čo je znázornené || na relačnom diagrame).
Návod: Skúste použiť anotácie `@JoinColumn(nullable=false)` a/alebo `@ManyToOne(optional=false)`.

b) Prepojovací stĺpec vo vygenerovanej tabuľke KNIHA sa volá VYDAVATEL (nie VYDAVATEL_ID).
Návod: použite anotáciu `JoinColumn(name="VYDAVATEL")`

Cvičenie 4.V projekte p32uniNN z prednášky implementujte nasledovné metódy:

1. `List<Osoba> nájdiAutorov(Long idKnihy)`
   Metóda dostane `id` knihy vyhľadá ju a vráti zoznam jej autorov. Ak kniha neexistuje, vráti `null`.
2. `int pocetKnih(String menoAutora)`
   Metóda dostane meno osoby a vráti počet kníh, ktorých je autorom. Ak osoba s daným menom neexistuje, vráti 0.
3. `List<Kniha> nájdiKnihy(String menoAutora)`
   Metóda dostane meno osoby, vyhľadá a vráti zoznam kníh, ktorých je autorom. Ak osoba s daným menom neexistuje, vráti `null`.

Pripravte vhodné testovacie dáta a otestujte si metódy pomocou jednoduchého programu.

Cvičenie 5.V projektoch p33uniN1 aj p34uni1N z prednášky implementujte metódu:

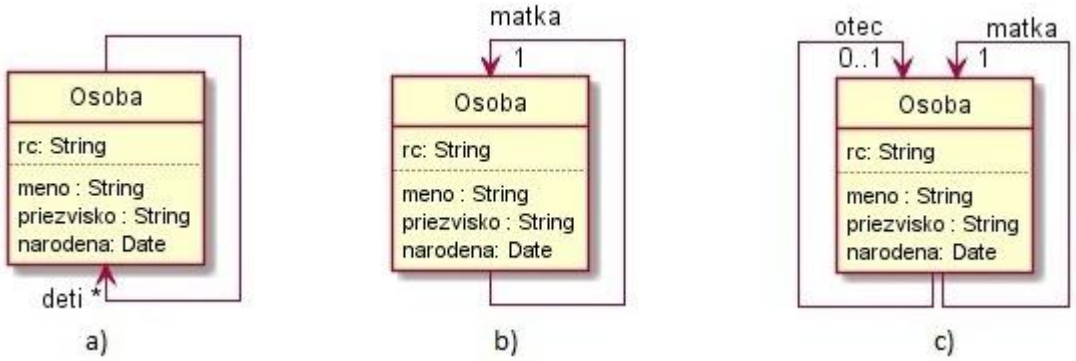
```
void pridajKnihu(String nazov, String adresa)
```

Metóda dostane názov knihy a adresu firmy, ktorá je jej vydavateľom a vloží ju do databázy, pričom:

- najprv overí či kniha s daným názvom, resp. vydavateľstvo s danou adresou už v DB existujú
- ak vydavateľstvo s danou adresou v DB ešte neexistuje, vytvorí ho,
- ak kniha s daným názvom v DB ešte neexistuje, vytvorí ju a priradí jej zadané vydavateľstvo (zaradí ju medzi publikácie vydavateľstva).
- ak kniha s daným názvom už v DB existovala neurobí nič.

Pripravte vhodné testovacie dáta a otestujte si metódu pomocou jednoduchého programu.

Cvičenie 6
Vzťahy medzi deťmi a rodičmi môžeme modelovať rôznymi spôsobmi, ako ukazujú tri rôzne verzie UML diagramu triedy `Osoba`:



Pre každú verziu:

- navrhnite a načrtnite, ako by vyzerala databáza odpovedajúca UML diagramu.
- implementujte entitnú triedu podľa UML diagramu a použite JPA anotácie tak, aby z nej JPA vygenerovalo databázu podľa vášho návrhu.

Implementujte tiež vhodný program na testovanie.